

IC Lab Formal Verification

Bonus Quick Test

2025 Fall

Name: 徐松廷

Student ID: 314510136

Account: iclab095

(a) Formal Verification

1. What is Formal verification?

formal verification 使用數學與邏輯的方法，證明設計在所有可能的輸入與所有可達狀態下，是否滿足指定的性質，會透過窮舉的方式，把所有可能的 input 組合全部嘗試一遍，確保沒有遺漏，在驗證過程中，會測試所有 input ports 與 undriven wire 可能的輸入組合；並且測試所有 uninitialized 的 register 所有可能的數值，以確保整個電路不存在任何 corner case。

在檢查過程中會將所有違反 assertion property 的情況以及所有被 hit 的 cover property 紀錄，最後整理並輸出成驗證結果(驗證結果的部分，Assert 的勾勾代表在所有情況該 assert property 都不會被違反，如果是交叉代表該 assertion 在某些情況會被違反，而 cover 的勾勾代表該 cover property 有被打中，又又代表沒有被成功 hit。)

2. What's the difference between **Formal** and **Pattern** based verification? And list the pros and cons for each.

A. difference between Formal and Pattern based verification

Formal verification 使用數學與邏輯的方法，證明設計在所有可能的輸入與所有可達狀態下，是否滿足指定的性質，透過窮舉所有可能的輸入與狀態，理論上可以完整覆蓋所有情況，但驗證速度較慢且隨設計複雜度增加而增加。Pattern-based verification 則是透過撰寫測試模式逐一驗證，速度快且可控，但可能漏掉某些 corner case。

需要嚴格遵守 protocol 的電路行為，適合使用 Formal Verification；有大量運算的電路，則較適合使用 pattern-based verification，以下為比較表格：

Item	Formal Verification	Pattern-based Verification
Method	Exhaustively explores all possible inputs and states	Tests one input pattern at a time
Coverage	Complete (no missed corner cases)	Limited; depends on test patterns
Speed	Slow; grows with design complexity	Fast and controllable
Effort	Little manual test writing	Requires manual/random/corner-case patterns
Weakness	Not scalable to very large designs	May miss untested corner cases

B. And list the pros and cons for each

formal verification: 透過數學方法，窮舉電路中所有 input ports 及所有 wire 的 value 組合。

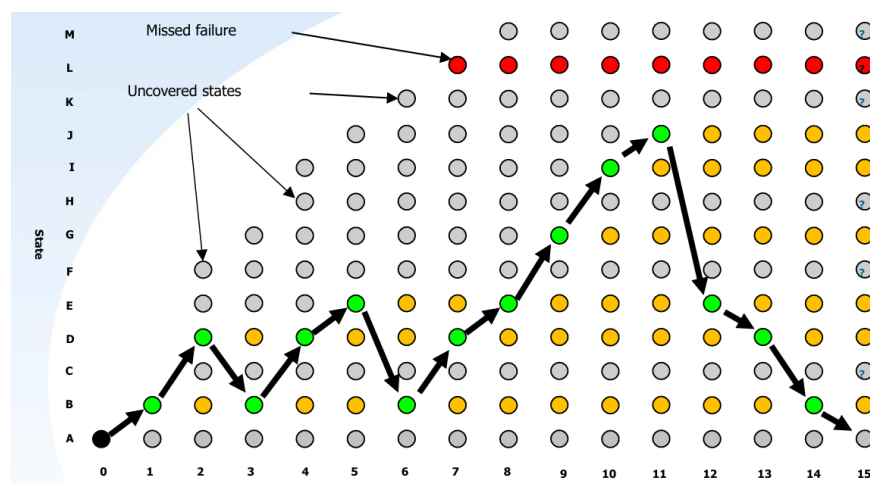
formal verification 優點: 能完整覆蓋所有可能情況，不需要擔心有任何 corner case 會被遺漏；通過 formal verification 的電路，在理論上可以視為已完整驗證。

formal verification 缺點: 因為必須窮舉所有可能性，驗證時間通常比 pattern verification 長得多，而且會隨著 design 的複雜度快速上升，可能難以在大型系統設計中使用，且需要額外的 EDA 工具協助進行。因此 formal verification 仍無法取代 pattern verification

pattern verification: 則是透過撰寫 pattern，每次僅餵入一組 input 組合來進行測試。它包含 random test 以及針對 corner case 所設計的測試。

pattern verification 優點: 驗證速度快，且能自行控制 pattern 的數量與覆蓋範圍，易於使用及實作。

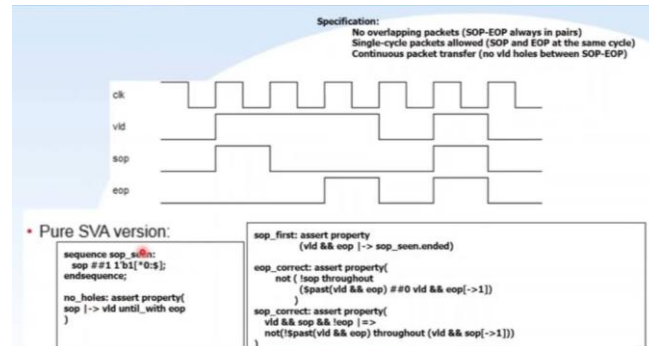
pattern verification 缺點: 無法保證窮舉所有 corner case，因此必須額外設計特定 pattern，以盡量提升整體驗證的完整性，下圖（引用自講義）說明了 pattern verification 可能在某些情況下遺漏特定的 input 組合與 input path。。



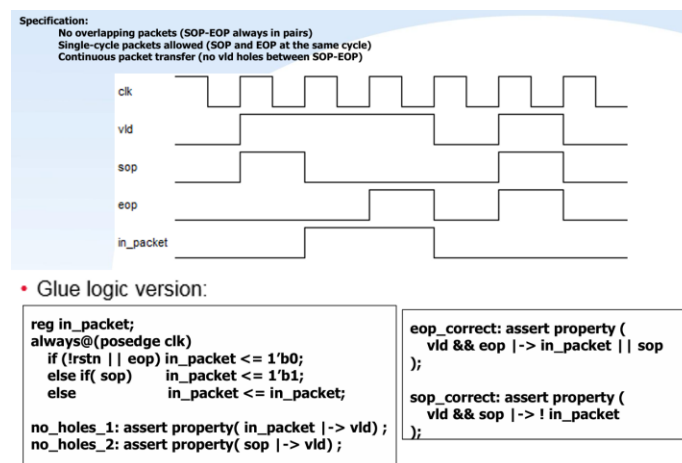
(b) Glue Logic

1. What is glue logic?

Glue logic 是利用額外的訊號來描述經常出現的行為訊號，以達到簡化邏輯的目的。例如以下講義中的範例，使用一般的 SVA 寫法，每個 assert property 中描述的邏輯會相對複雜且冗長；



但如果使用 glue logic，並多宣告一些訊號（例如 in_packet），就可以用這些變數去描述原本複雜的邏輯，從而簡化語法，而功能則完全不變。且在大多數情況下，這些 glue logic 並不會被編譯器合成，因此不會對原本的設計造成負擔。



2. Why will we use glue logic to simplify our SVA expression?

使用 Glue Logic 可以將原本複雜的 assert property 進行簡化，同時提升在其他模組中重複使用的便利性，並增加程式的可讀性。對於大型系統整合與團隊協作而言，這種做法尤為有利。此外，Glue Logic 也可能降低模擬與驗證過程中的運算成本，因為它透過將可重複使用的行為描述抽象成單一變數，進而簡化原本複雜的 assert property，同時也提升 SVA 的可讀性。

(c) Coverage

1. What is the difference between **Functional coverage** and **Code coverage**?

Functional Coverage 用於評估電路或 RTL 程式中所有輸入訊號的可能組合是否已被完整驗證。它透過檢查設計者所定義的 assert property 以及 cover group，以確認電路的功能與行為是否符合預期。

Code Coverage 則是針對 RTL 程式的執行情況進行檢查，評估例如 branch、statement 或 expression 是否已被完整執行。它主要關注程式邏輯的覆蓋情況，而不直接驗證電路功能的正確性，可能無法保證電路功能正確。

Aspect	Formal Functional Coverage	Code Coverage
Focus	Verifies circuit functionality and behavior	Checks RTL code execution
Checks	All possible input combinations, functional correctness	Branches, statements, and conditions executed
Method	<code>assert</code> , <code>property</code> , <code>cover</code> , <code>group</code> , formal verification	Simulation-based code execution tracking
Purpose	Ensure correct circuit behavior	Ensure code coverage and test completeness
Limitation	Requires well-defined assertions/cover groups; may be computationally intensive	Does not guarantee functional correctness

2. What's the meaning of 100% code coverage, could we claim that our assertion is well enough for verification? Why?

即使達到 100% 的 Code Coverage，也無法保證設計的功能正確性，因為 Code Coverage 僅表示 RTL 程式碼中的每一行與每一個分支都曾被執行過，並不代表設計本身的行為符合規格，可能存在並未涵蓋在程式中的電路行為及狀態並不會被 code coverage 檢查到，或是存在沒有被考慮到的 corner case。

要確認 design 的功能是否正確，仍需依靠 Functional Coverage，以驗證設計的所有預期行為與輸入情境是否都已被完整測試。

(d) What is the difference between COI coverage and proof coverage for realizing checker's completeness? Try to explain from the **meaning**, **relationship**, and **tool effort** perspective.

1. Meaning

在 formal testbench analysis 中，驗證可以分為 stimulus coverage 與 checker coverage 兩大類。Stimulus coverage 主要檢測輸入空間（input space），確保各種可能的輸入組合都被測試到；而 checker coverage 則著重於 property space，也就是驗證 property 或 assertion 是否完整被檢查。

在 checker coverage 之下，又可以細分為 COI coverage 與 proof coverage。COI coverage（Cone of Influence 覆蓋率）用來偵測所有可能影響該 property 或 assertion 的電路訊號，重點在於訊號的廣度；而 proof coverage（證明覆蓋率）則使用 formal engine 進行形式化驗證，僅包含 assertion 內實際使用到的訊號，強調對狀態空間的深度分析。

COI coverage 與 proof coverage 結合起來形成完整的 checker coverage，用於彌補 stimulus coverage 無法保證設計 100% 正確性的不足。簡單來說，COI coverage 關注的是設計中所有可能影響 assertion 的訊號，而 proof coverage 則關注 assertion 本身實際包含的訊號並對其進行完整形式化證明。

2. Relationship

COI (Cone of Influence) 包含程式碼中所有可能影響該 assertion 的訊號，而 Proof 則指 assertion 中實際被使用或包含的訊號。因此，Proof Coverage 是 COI Coverage 的子集。COI Coverage 涵蓋所有潛在影響訊號，而 Proof Coverage 所涉及的實際訊號必然包含在其中。換句話說，Proof Coverage 是 COI Coverage 的一個子集，若電路中所有與 assertion 相關的訊號都有實際影響，則 Proof Coverage 將等於 COI Coverage。

3. Tool effort

由於 Proof Coverage 需要透過形式驗證來計算，相較於 COI Coverage，其執行會消耗更多的運算資源與時間。Proof Coverage 代表真正影響 assertion 的訊號，因此必須使用 formal engine 進行驗證，因此所需的驗證時間和資源都比 COI Coverage 高。

(e) What are the roles of **ABVIP** and **Scoreboard** separately? Try to explain the **definition**, **objective**, and the **benefit**.

1. Definition

A. ABVIP (Assertion Based Verification Intellectual Properties):

ABVIP 是一套預先撰寫好的驗證 IP，內含完整的測試與檢查用的 assertions，通常用於驗證設計中特定的功能或協定，是一種高可靠性的驗證工具，能提升驗證品質並縮短開發時間。包括 checker 與 RTL，用於檢查設計是否符合特定 protocol 或 interface 的規範。

B. Scoreboard:

Scoreboard 是一種驗證監視環境，用於追蹤並觀察設計輸入值、預期輸出值與實際輸出值之間的關係，以驗證設計的正確性。

2. Objective

A. ABVIP (Assertion Based Verification IP) :

提供設計者一套完善的工具，用以檢查既有的協定 (protocol)。ABVIP 模擬與 DUT 的互動介面，檢查協定或介面的正確性，提升驗證品質，並縮短開發時間。

B. Scoreboard :

用於檢查設計層級上輸入與輸出的數值一致性。它會比較 DUT 的輸出訊號與參考模型 (reference model) 或 golden output 的結果，以確保輸出正確。

3. Benefit

A. ABVIP (Assertion-Based Verification IP)

ABVIP 具有高度的便利性，能有效簡化 testbench 的開發流程，降低整體設計複雜度與人力成本。透過內建的 assertion 與協定檢查機制，ABVIP 能自動化地確認通訊協定 (protocol) 的正確性，避免人工撰寫檢查邏輯時可能產生的疏漏，進而大

幅降低錯誤發生率。同時，它能節省工程師設計 assertion 的時間，使驗證流程更具效率與可靠性。

B. Scoreboard

Scoreboard 以高階抽象層級進行行為比對與結果驗證，能在不陷入過度細節的情況下協助除錯，從而有效減少 state-space 的複雜度。由於其運作邏輯通常比底層 RTL 更具可讀性與可管理性，因此能提升驗證與問題追蹤的便利性。此外，經過 formal 驗證方法的最佳化，Scoreboard 能進一步縮小驗證空間，降低使用門檻，使整體驗證流程更高效。